填充规则（fillRule）

`fill(fillRule)`

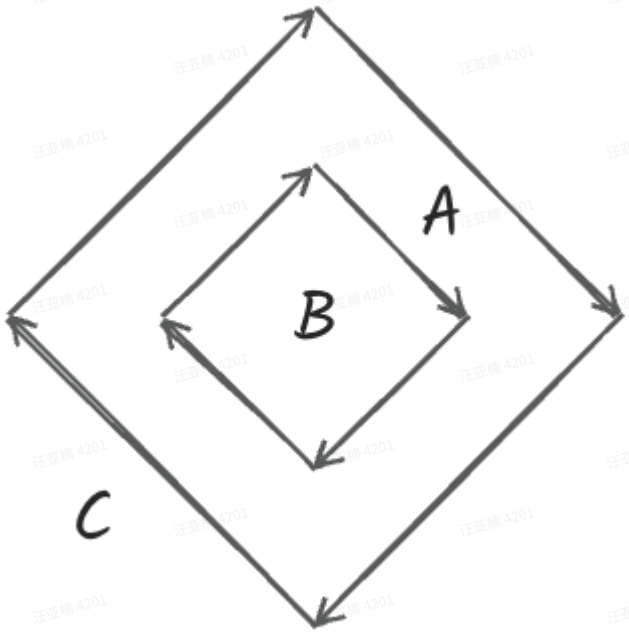
填充当前路径区域，如果路径未闭合则尾部自动连接起始点形成闭合区域。

如果你有仔细留意，`fill` API有一个可选参数 `fillRule`，它可以设置以何种规则去执行本次路径区域的填充操作，它有两个参数：

① nonzero，非零环绕规则（默认）

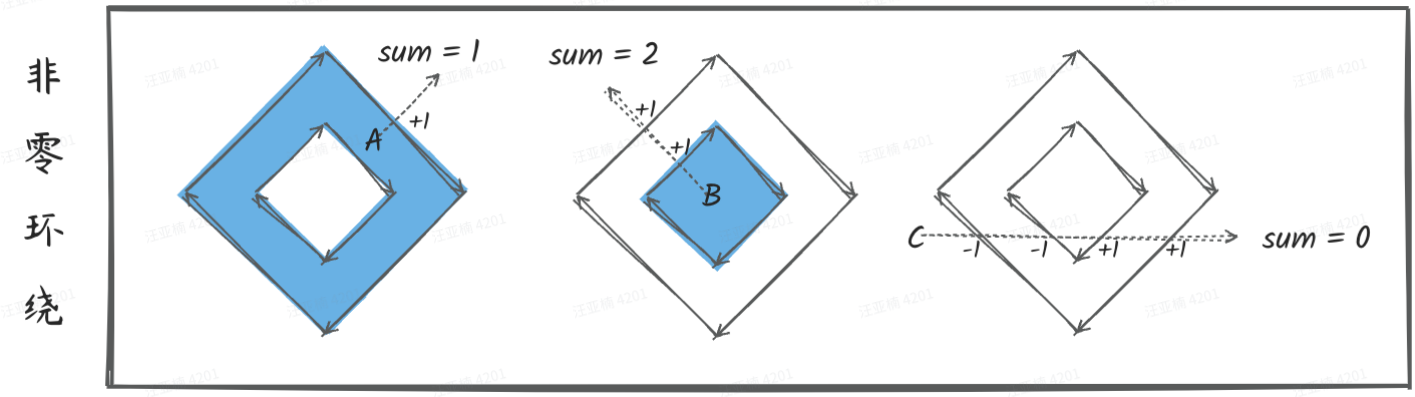
② evenodd，奇偶环绕规则

1）什么是环绕规则？

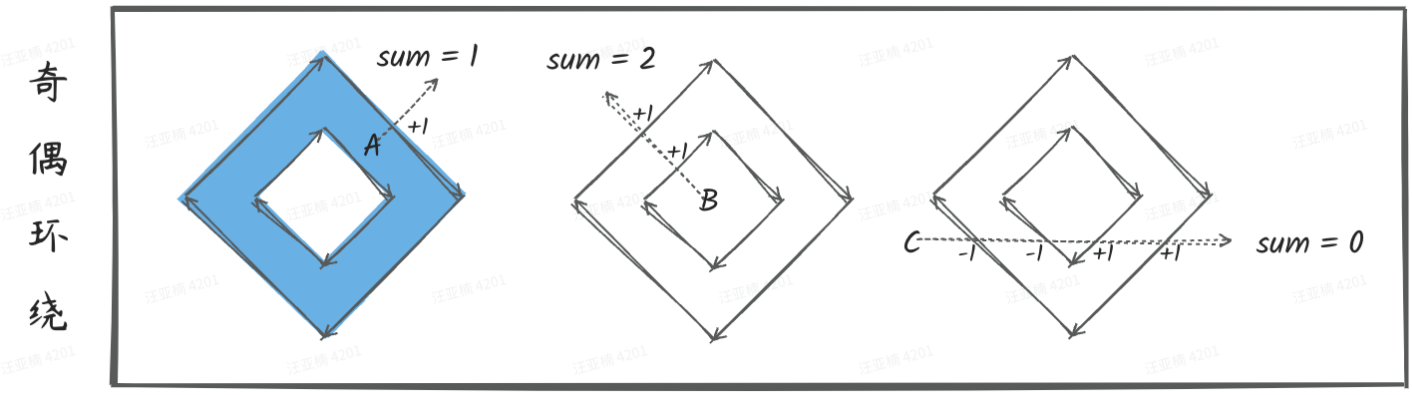


假如我们有一段路径，组合成两个方形。

这个路径将整个画布分割成三个区域，我们在各个区域中随机找出A、B、C三个点，并以该点开始向随机方向画一条射线，该射线与路径的相交时，如果所相交的路径是顺时针方向则+1，反之-1，如下图：



当使用“非零环绕”规则时，最终计数值不为0时则属于路径内部区域，会被填充。



当使用“奇偶环绕”规则时，最终计数值为奇数时属于路径内部区域，会被填充。所以这里只有A区域会被填充。

2）随机点？随机射线？最终计数值？

相信大家在听我讲上面知识的时候会有很多疑惑，比如：该如何判定随机点、射线方向又如何决定，怎么知道是最终计数值...所以这里就提供一个便于理解的过程：

① 如何判定随机点

你创建了一段路径，当调用 `fill` 方法后，一开始整个画布中所有的像素点都是不清楚其是否处于路径内部区域的，既然如此，我们便可以大胆地依次将每个像素都作为我们的“随机点”来判断其是否处于路径的内部区域，如果是则后续参与填充。

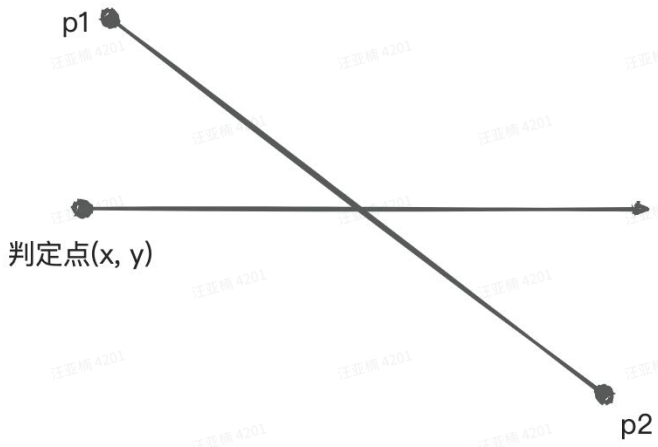
在实际的图形渲染过程中，画布肯定是不会对每个像素点都做一条射线来判断是否在路径内部。这种方法理论上是可行的，但实践中会因大量的计算而变得非常低效。所以实际的图形渲染会使用更加高效的算法去确定哪些像素在路径内部，但这已经超出了本文的范围，就不再深入，更多可自行查阅。

② 画哪个方向的射线

这里其实是可以自由决定是绘制一条水平还是垂直的射线，当然斜线也可以的，为了便于计算我们选择一条水平线来作为我们需要的射线。

③ 如何判定相交

当我们依次遍历路径中的点时，便可以通过前后两点来形成线段（这里不考虑曲线），有了这条线段，射线与其相交关系自然而然也是可以通过数学计算求出。同样的相交的线段我们也可以通过数学计算求出，但这里不展开了，有兴趣看下面文章：深入canvas/svg的布尔运算（Martinez法）



这里提一句，事实上只有非零环绕才需考虑路径方向，奇偶环绕只需要计算交点数，交点总数为奇数时计算值必然也是奇数，某些情况下可以减少运算消耗。

④ 怎样算是计数完成

当所有路径点遍历完成后，也意味着我们已经判定完成了该射线与所有路径点形成的线段之间的相交关系，最后得到的计数值则可视做最终计数值。

最后我们便可根据“填充规则”设置来判定其是否在内部区域而决定其是否参与填充。

3） `isPointInPath` & `isPointInStroke`

上面只是简单模拟如何判断是路径区域内的点都十分复杂，如果我们在画布应用开发时也需要做这个判断那必然十分麻烦的。但好在 `Canvas` 为开发者提供了两个十分强大的原生API：

① `isPointInPath(x, y, fillRule)` 判断点 `(x, y)` 是否在路径区域内。

② `isPointInStroke(x, y, fillRule)` 判断点 `(x, y)` 是否在路径的描边线上。

这两个API在区域选择和碰撞检测中可以发挥出非常大的作用。

4）了填充规则背后的原理的意义

① 你可以使用它进行更加精确的图像绘制，同时也可以实现一些复杂的图形，下面示例使用了 `fillRule` 来绘制月牙：

https://codepen.io/huanjinliu/embed/oNQJOWy

思考：如果不使用 `fillRule` 你有其他实现方法吗？

② 你可以通过合理地使用路径和设置填充规则来提高画布的绘制性能，比如下面绘制PS格子背景的例子，合理使用 `fillRule` 速度快了整整10倍。

https://codepen.io/huanjinliu/embed/LYXMawO?editors=1010

思考：下面程序中，路径形成的交集区域实际上参与了几次像素计算？

```
const ctx = canvas.getContext('2d');

ctx.beginPath();
ctx.rect(0, 0, 100, 100);
ctx.rect(50, 50, 100, 100);

ctx.fill();
```

解答：

如果你不明白路径的填充过程，你可能会觉得这里是先绘制了一个矩形后再绘制另一个矩形，所以会认为交集区域应该参与了两次像素计算。

但实际上浏览器会先通过整体路径判断该区域是否是路径内部区域，是则填充，所以整个流程中该区域的像素参与一次计算。